



UMCS

**UNIWERSYTET MARII CURIE-SKŁODOWSKIEJ
W LUBLINIE**

Wydział Matematyki, Fizyki i Informatyki

Kierunek: Informatyka

Jakub Krać

nr albumu: 308178

**Implementacja algorytmu uczenia
maszynowego w sterowaniu agentem w
grze platformowej**

**Implementation of a machine learning algorithm in controlling
an agent in a platform game**

Praca licencjacka

napisana w Katedrze Neuroinformatyki i Inżynierii Biomedycznej
pod kierunkiem dr Anny Gajos-Balińskiej

Lublin rok 2024

Spis treści

Wstęp	1
1. Uczenie maszynowe w grach platformowych	3
1.1. Gry platformowe	3
1.2. Agent	4
1.3. Uczenie maszynowe przez wzmacnianie	5
1.4. Algorytmy sztucznej inteligencji	6
2. Projektowanie i implementacja	9
2.1. Kluczowe elementy	9
2.2. Wybór i przygotowanie silnika	9
2.3. Menu gry	10
2.4. Agent	12
2.5. Przeciwnik	13
2.6. Przeszkody	15
2.6.1. Lawa	15
2.6.2. Czarna materia	15
2.6.3. Kolce	16
2.6.4. Emiter lawy	16
2.7. Kamera	18
2.8. Poziomy	19
2.8.1. Poziom 0	20
2.8.2. Poziom 1	20
2.8.3. Poziom 2	21
2.8.4. Poziom 3	21
2.9. Implementacja algorytmu	22
2.10. Konsola	25
3. Testowanie aplikacji	27
3.1. Scenariusz testów	27
3.2. Testy	27
3.3. Wnioski	28
Podsumowanie	31

Wstęp

Rozwój maszyn komputerowych umożliwił utworzenie gier platformowych 2D, które pomimo swojej prostoty wciąż są popularne na rynku. Za pomocą uczenia maszynowego wspomniane gry mogą zoptymalizować swoją trudność, czy też proceduralnie tworzyć poziomy. W tej pracy uczenie maszynowe zostanie wykorzystane w celu przejścia poziomów gier platformowych. Takie podejście może być wykorzystane w celu szukania błędów w grach czy też odkrywania nieznanych możliwości, które mogłyby umknąć uwadze człowieka. Dzięki temu gry platformowe mogą stać się jeszcze bardziej angażujące i satysfakcjonujące.

W pierwszym rozdziale zostaną omówione podstawowe pojęcia związane z uczeniem maszynowym oraz grami platformowymi. Z kolei drugi rozdział przedstawi proces implementacyjny aplikacji oraz algorytmu uczenia maszynowego. W ostatnim rozdziale zostanie przeprowadzona analiza oraz skuteczność działania algorytmu w utworzonej aplikacji.

Słownik pojęć

Tileset (zestaw kafelków) to zbiór małych obrazków (kafelków), które używane są do tworzenia większych obrazów poprzez łączenie ich razem.

Tilemap (mapa kafelków) to struktura danych, która przechowuje informacje o rozmieszczeniu kafelków. Pojęcie to jest ściśle związane z **tilesetem**.

Kolizja jest interakcją między obiektami, która zachodzi gdy ich pozycje w przestrzeni nachodzą na siebie. Zderzenie to wymaga określonej reakcji w grze.

Hitbox jest kolizją, która w momencie spotkania dwóch lub większej liczby obiektów najczęściej zadaje obrażenia.

GUI jest graficznym interfejsem użytkownika (Graphical User Interface). Umożliwia interakcję człowieka z komputerem poprzez wykorzystanie elementów graficznych, takich jak ikony czy przyciski.

MarginContainer jest komponentem używanym w narzędziach służących do tworzenia interfejsów użytkownika takich jak **GUI** (Graphical User Interface). Jego główną funkcją jest dodawanie odstępów wokół swojej zawartości.

HBoxContainer jest komponentem służącym do tworzenia interfejsów użytkownika. Jego funkcją jest rozmieszczenie elementów w układzie poziomym.

VBoxContainer jest komponentem podobnym do **HBoxContainer**. Różni się tym, że umieszcza elementy w układzie pionowym.

Camera2D jest komponentem, który umożliwia kontrolowanie widoku 2D.

Node:Control jest podstawową klasą w GODOT, która zawiera właściwości i metody ułatwiające tworzenie i zarządzanie interfejsem użytkownika.

AnimationPlayer to węzeł w GODOT, który umożliwia tworzenie, zarządzanie oraz odtwarzanie animacji.

Pixelart polega na tworzeniu obrazów przy użyciu pikseli jako podstawowych elementów graficznych. Technika pixelartu cechuje się użyciem ograniczonej palety kolorów oraz ręcznym ustawieniem każdego piksela na planszy.

Side-scrolling jest to sposób, w którym kamera śledzi postać gracza lub jest zablokowana na konkretnej wysokości ekranu, przesuając się wraz z postępem postaci w lewo lub prawo.

Rozdział 1

Uczenie maszynowe w grach platformowych

Celem tego rozdziału jest przedstawienie podstawowych pojęć związanych z uczeniem maszynowym, omówienie gier platformowych oraz krótkie opisanie algorytmów mogących sterować postacią gry komputerowej w celu przejścia poziomu.

1.1. Gry platformowe

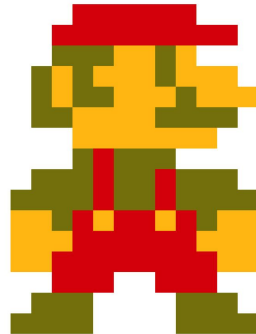
Wraz z rozwojem komputerów w drugiej połowie XX wieku na rynku zaczęły się pojawiać gry platformowe, wyróżniające się następującymi cechami:

- rozgrywka projektowana jest wokół głównego bohatera,
- najczęściej główną umiejętnością bohatera jest skok,
- bohater w trakcie przechodzenia poziomu napotyka różne przeszkody (ruszające się platformy, przeciwników czy lawę),
- bohater zostaje umieszczony w świecie składającym z poziomów o różnym stopniu trudności,
- w celu przejścia poziomu gry bohater musi najczęściej dostać się do oznaczonego miejsca, na przykład przez flagę.

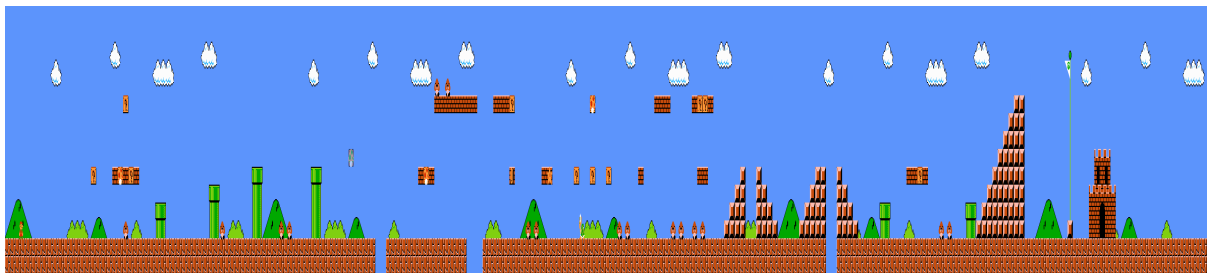
Jeszcze kilkadziesiąt lat temu komputery nie były tak samo powszechne jak dziś i znaczna większość gier była produkowana na konsole takie jak Sega Mega Drive czy Super Nintendo Entertainment System.

Jedną z najbardziej popularnych serii gier platformowych w XX wieku była seria *Super Mario*, której pierwsza gra została wydana przez Nintendo w 1985 roku pod tytułem *Super Mario Bros.* Głównym bohaterem tej gry jest tytułowy Mario (przedstawiony na Rysunku 1.1). Za pomocą sterowania ze strony użytkownika Mario przechodził poziom gry (Rysunek 1.2), napotykając na swojej drodze charakterystyczne dla tego gatunku gier przeszkody.

W tamtym okresie powstały także produkcje takie jak *Megaman* (1987), *Duke Nukem* (1991), *Sonic The Hedgehog* (1991) i wiele innych.



Rysunek 1.1: Mario (źródło: <https://blogs.loc.gov/now-see-hear/2023/08/from-the-national-recording-registry-super-mario-1985>)



Rysunek 1.2: Początkowy poziom gry Mario (źródło: <https://www.reddit.com>)

Skuteczność Mario zależała wyłącznie od sterującego nim użytkownika. Wiele lat później nastąpił znaczny rozwój sztucznej inteligencji, który umożliwił przechodzenie poziomów gier platformowych za pomocą komputera. Wraz z tym postępem narodziło się pojęcie *agenta*, które zostanie omówione w następnym podrozdziale.

1.2. Agent

Microsoft definiuje agenta sztucznej inteligencji jako „kod lub mechanizmy, które działają w celu osiągnięcia wstępnie określonych celów” [1]. Warto dodać, że agent może postrzegać środowisko za pomocą sensorów i wykonywać czynności za pomocą mechanizmów wykonawczych. Przykładem takiego agenta może być program komputerowy, którego sensorem jest klawiatura, a mechanizmem wykonawczym ekran. W celu zapewnienia odpowiedniego działania takiego agenta często stosuje się strukturę PEAS [2]:

- pomiar wydajności (**P**erformance measure),
- środowisko (**E**nvironment),

- mechanizmy wykonawcze (**A**ctuators),
- sensory (**S**ensors).

Wyróżnia się cztery rodzaje agentów [3]:

- Agent refleksowy – wykonuje czynności wyłącznie na podstawie najświeższych obserwacji oraz wbudowanych reguł.
- Agent refleksowy z modelem świata – w odróżnieniu od swojego poprzednika posiada wiedzę o środowisku, która pozwala uzupełnić brakujące informacje o otoczeniu.
- Agent celowy – posiada wyznaczony cel, który określa jakie stany są oczekiwane. Agent ten w odróżnieniu od poprzedników bierze pod uwagę przyszłość.
- Agent użyteczny – jeżeli będzie mógł osiągnąć cel wieloma sposobami, to być może wybierze najlepszy z nich.

W przypadku gier platformowych agentem najczęściej jest sterowalna postać bądź przeciwnik. W celu odpowiedniego działania agenci potrzebują mechanizmów pozwalającym im wykonywać czynności w otaczającym ich środowisku. Potrzebna jest im odpowiednia forma uczenia się, która w kontekście informatycznym określana jest uczeniem maszynowym. Pojęcie to posiada trzy główne nurty: uczenie nadzorowane, uczenie nienadzorowane oraz uczenie maszynowe przez wzmacnianie.

Uczenie maszynowe wyróżnia się tym, że na podstawie jakiś danych, algorytm ma znaleźć odpowiednie reguły. Proces ten polega na analizie dużych ilości informacji i automatycznym wnioskowaniu, co pozwala systemowi adaptować się do zmieniającego się środowiska i dokonywać przewidywań lub podejmować decyzje na podstawie zebranych danych [4].

W celu nauczania agenta przechodzenia poziomów gier platformowych częstym wyborem jest uczenie maszynowe przez wzmacnianie.

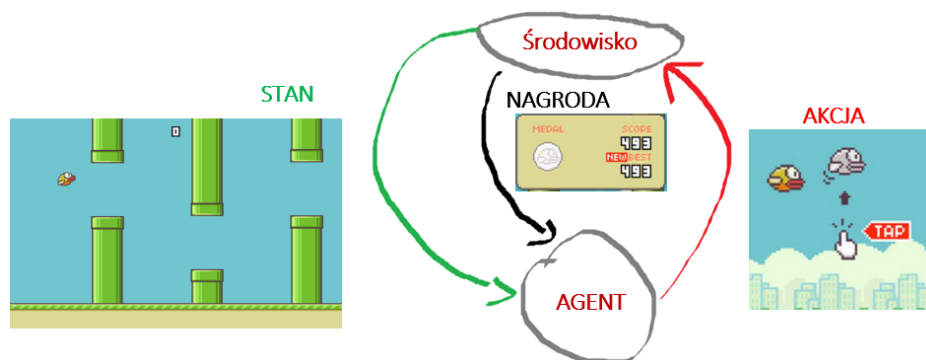
1.3. Uczenie maszynowe przez wzmacnianie

Agent wykorzystujący uczenie maszynowe przez wzmacnianie będzie wykonywał różne czynności i obserwował ich rezultaty. Za każde poprawne działanie zostanie nagrodzony, a w przypadku niepowodzenia otrzyma karę. Z tego wynika, że będzie uczył się na bazie swoich doświadczeń podobnie jak człowiek. Warto zaznaczyć, że młodzi ludzie zazwyczaj uczą się w sposób nadzorowany w większości przypadków. Nie bez powodu termin uczenia przez wzmacnianie wywodzi się z psychologii [5]. Wyróżnia się dwa rodzaje wzmocnień: pozytywne (zwiększa prawdopodobieństwo, że zachowanie będzie się powtarzać) i negatywne (zmniejsza prawdopodobieństwo, że zachowanie będzie się powtarzać). Podczas procesu uczenia agenta wyróżnia się cztery elementy:

- stany początkowe,

- stan nowy,
- akcje,
- nagrody.

Ilustruje to Rysunek 1.3.



Rysunek 1.3: Elementy uczenia maszynowego przez wzmacnianie (źródło: <https://mirosławmamczur.pl/czym-jest-uczenie-maszynowe-i-jakie-sa-rodzaje>)

Wyróżnia się dwa rodzaje zadań: zadania epizodyczne (proces uczenia będzie trwał, aż do spełnienia kryteriów końcowych na przykład dotarcie do punktu końcowego poziomu) i zadania ciągłe (proces uczenia będzie trwał do momentu, w którym programista zadecyduje, żeby przerwać szkolenie) [6]. W procesie szkolenia agenta istnieją dwie kluczowe koncepcje, które określają sposób, w jaki agent podejmuje decyzje w interakcji ze środowiskiem:

- Eksploracja to zbieranie większej ilości informacji o otoczeniu. Głównym celem tej czynności jest poznanie i zrozumienie pełnego zakresu możliwości, jakie oferuje środowisko. Takie działanie pozwala agentowi nauczyć się optymalnej strategii podejmowania decyzji.
- Eksploatacja wykorzystuje znane informacje lub strategię w celu maksymalizacji nagród. Głównym celem tego procesu jest wybieranie działań, które jak dotąd przynosiły najlepsze wyniki.

1.4. Algorytmy sztucznej inteligencji

W celu wykonania zadania agenta (przejścia poziomu gry) mogą zostać wykorzystane następujące algorytmy: Q-Learning, algorytm genetyczny czy Deep-Q-Networks.

Q-Learning. Algorytm ten oparty jest na wartościach. Skupia się na optymalizacji funkcji wartości ($Q(s, a)$, s – stan, a – akcja). Model trzyma każdą wartość w tablicy Q w celu przechowania wiedzy, którą agent zdobył. Umożliwia to podjęcie akcji w określonych stanach, aby maksymalizować przyszłe nagrody. Proces szkolenia zaczyna się od inicjalizacji. Agent na samym początku nie posiada

żadnej wiedzy. Z tego wynika, że początkowo tablica Q jest równa 0. W późniejszym etapie agent musi wybierać pomiędzy eksploatacją a eksploracją. Po podjęciu decyzji agent otrzyma nagrodę od środowiska, a następnie środowisko przejdzie do nowego stanu na podstawie podjętej akcji. Po wszystkich krokach w tablicy Q pojawiają się zaktualizowane wartości [7].

Deep-Q-Networks. Algorytm ten rozwija poprzednio omawiany Q-Learning o wykorzystanie głębokich sieci neuronowych, służących do aproksymacji funkcji $Q(s, a)$. Algorytm składa się z następujących elementów:

1. **Replay buffer** przechowuje doświadczenia agenta w postaci (s, a, r, s') , r – nagroda, s' – następny stan. Stosowanie takiego buforu pozwala na wielokrotne wykorzystanie doświadczeń podczas uczenia, poprawiając stabilność i skuteczność.
2. **Q-Network** to głęboka sieć neuronowa, która w warstwie wejściowej przyjmuje stan gry i zwraca wartość akcji w warstwie końcowej.
3. **Celowa sieć Q-Network** jest kopią **Q-Network**. Wykorzystywana jest do oceny przyszłych wartości akcji podczas obliczania oczekiwanej wartości nagrody. Nie jest aktualizowana często, ale dzięki temu poprawia się stabilność uczenia.

Algorytm genetyczny. Algorytm ten inspirowany jest procesami zachodzącymi w przyrodzie takimi jak mutacje, dziedziczenie genetyczne czy krzyżowanie. Zazwyczaj składa się z następujących kroków [8]:

1. **Inicjalizacja populacji** – początkowa populacja rozwiązań (najczęściej ciąg binarny) jest losowo generowana.
2. **Selekcja** – wybierane są osobniki do reprodukcji na podstawie ich funkcji oceny.
3. **Krzyżowanie** – tworzone są nowe rozwiązania poprzez wymianę informacji genetycznej pomiędzy osobnikami.
4. **Mutacja** – niektóre cechy pewnych osobników zostają losowo zmienione. Zapewnia to różnorodność genetyczną.
5. **Ocena** – oceniana jest jakość każdego osobnika w populacji za pomocą funkcji przystosowania.
6. **Zastąpienie** – stara populacja jest zastępowana nową, wygenerowaną po przejściu przez poprzednie kroki.
7. **Warunki stopu** – algorytm działa, dopóki nie zostanie spełniony warunek definiujący jego zakończenie.

Rozdział 2

Projektowanie i implementacja

W tym rozdziale zostały omówione główne założenia projektu oraz został dokładnie opisany proces implementacyjny gry.

2.1. Kluczowe elementy

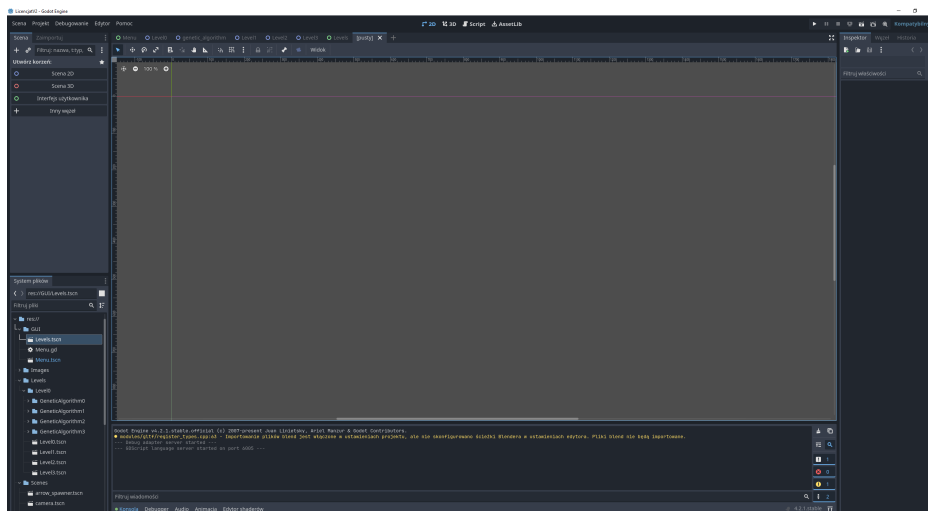
Celem pracy było stworzenie gry platformowej inspirowanej produkcjami z drugiej połowy XX wieku, w której agent za pomocą odpowiedniego algorytmu mógłby przechodzić poziomy gry. Konieczne zatem było stworzenie odpowiedniego środowiska, w którym został umieszczony agent, przeciwnicy oraz przeszkody. Konieczny był także wybór odpowiedniego silnika gier komputerowych.

Środowiskiem są zróżnicowane poziomy zbudowane tak, aby agent był w stanie przejść poziom poruszając się od punktu startowego za pomocą akcji skoku oraz ruchu w prawo do punktu oznaczonego przez flagę. Agent posiada pasek życia składający z procentów ($0\% \leq \text{śmierć}$, $>0\%$ zdolny do poruszania) i nie posiada zdolności ataku. Podczas swojej podróży napotka różne przeszkody takie jak lawa, ruszający się przeciwnicy, przepaście oraz kolce. Poziomy posiadają skalujący się poziom trudności, to znaczy poziom pierwszy powinien być najkrótszy i najłatwiejszy, a każdy następny coraz trudniejszy i dłuższy. Wraz z kolejnymi poziomami agent napotyka przeszkody, których wcześniej nie był w stanie napotkać. Poziomy zostały zaprojektowane tak, żeby przypominały podróż przez las.

Gra została wykonana w grafice 2D w stylu **pixelart** z bocznym rzutem kamery (**side-scrolling**) i była tworzona na systemie Windows 10.

2.2. Wybór i przygotowanie silnika

Gra została utworzona na silniku Godot w wersji 4.2.1-stable, który pozwala na tworzenie gier 2D, 3D oraz umożliwia programowanie za pomocą języków takich jak GDScript, C#, C++. Spośród tych trzech języków każdy kod gry został napisany w GDScript. Godot posiada licencję MIT, która daje użytkownikowi nieograniczone prawo do używania, kopiowania, modyfikowania oraz rozpowszechniania programu. Główne okno środowiska Godot zostało przedstawione na Rysunku 2.1.

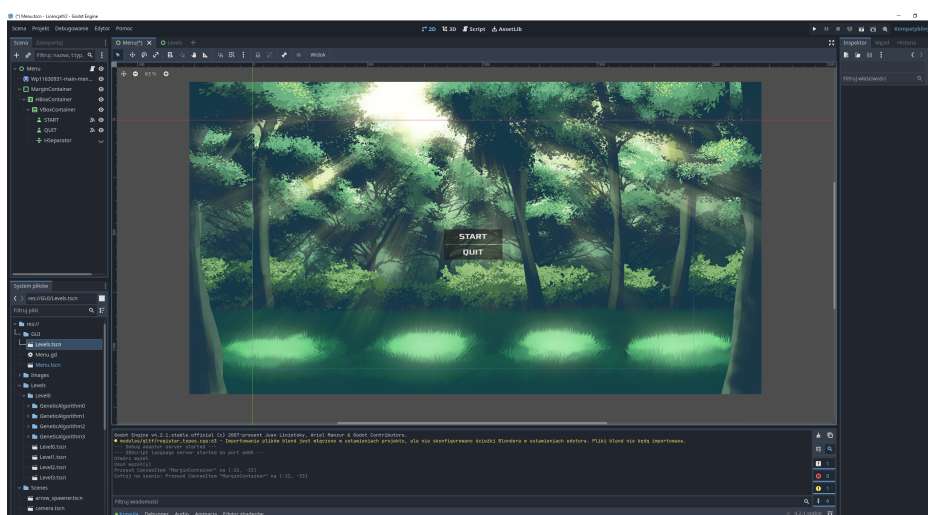


Rysunek 2.1: Główne okno Godot

Godot posiada Renderer, który renderuje grafikę w grach i aplikacjach. W trakcie tworzenia projektu można wybrać następujące tryby, w których może pracować Renderer: Przedni+, Mobilny i Kompatybilny. Gra korzysta z tej ostatniej opcji, ponieważ pozwala ona na najszybsze renderowanie prostych scen.

2.3. Menu gry

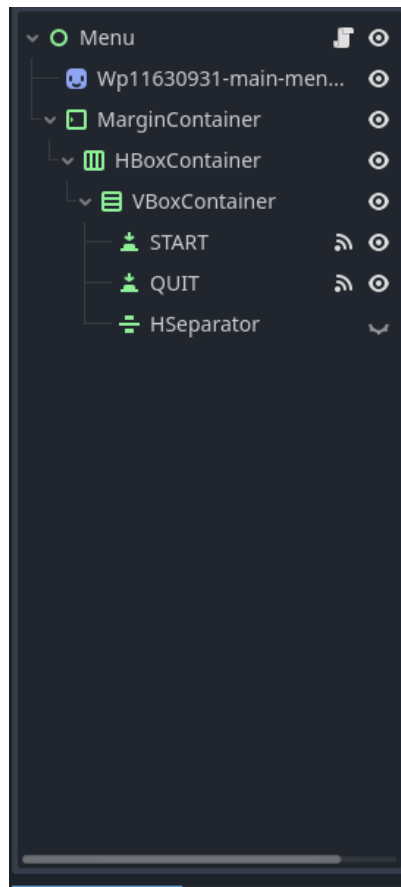
Menu gry zostało stworzone za pomocą elementów **Node:Control**, czyli specjalnych węzłów do tworzenia **GUI** (Graphical User Interface). Menu pokazane jest na Rysunku 2.2.



Rysunek 2.2: Menu panelu początkowego

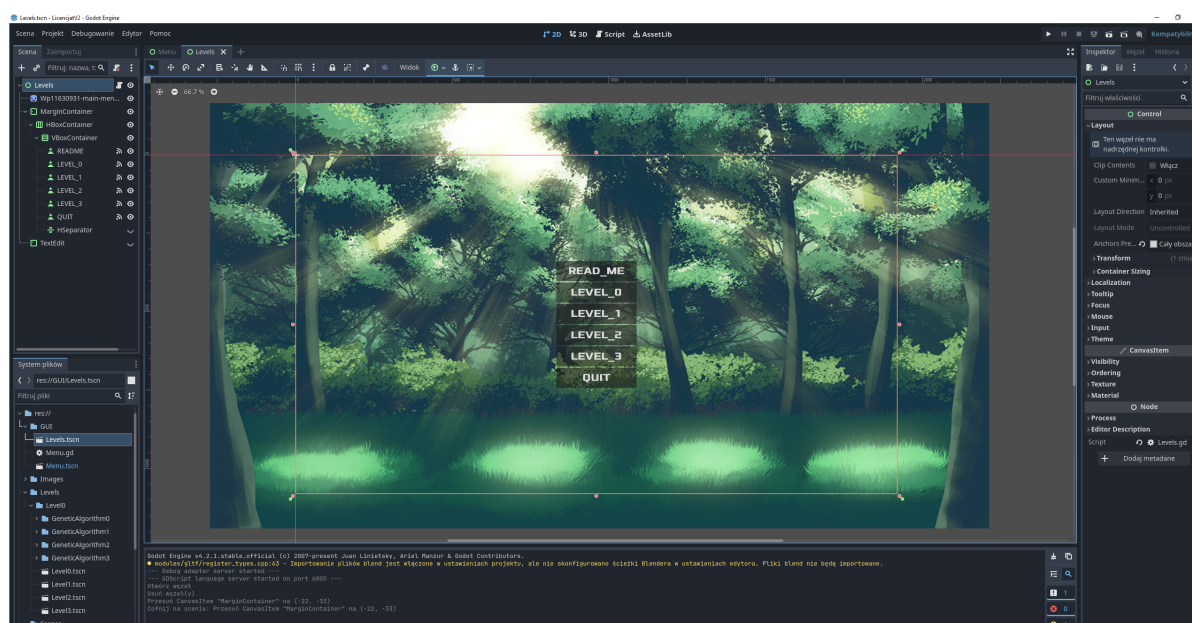
Na Rysunku 2.3 pokazane jest za pomocą jakich węzłów został wykonany panel początkowy. **MarginContainer**, **HBoxContainer**, **VBoxContainer** należą do **Node:Control** i odpowiadają za odpowiednie rozmieszczenie elementów.

START i **QUIT** są elementami połączonymi ze skryptem menu za pomocą sygnałów, czyli mechanizmów pozwalających na komunikację między obiektami. Po naciśnięciu przycisku **START** następuje przejście do panelu z poziomami gry, a przycisk **QUIT** wyłącza aplikację. Widoczne tło gry zostało wykonane przez użytkownika o pseudonimie sismcastar [9].



Rysunek 2.3: Zrzut ekranu przedstawiający węzły menu panelu początkowego

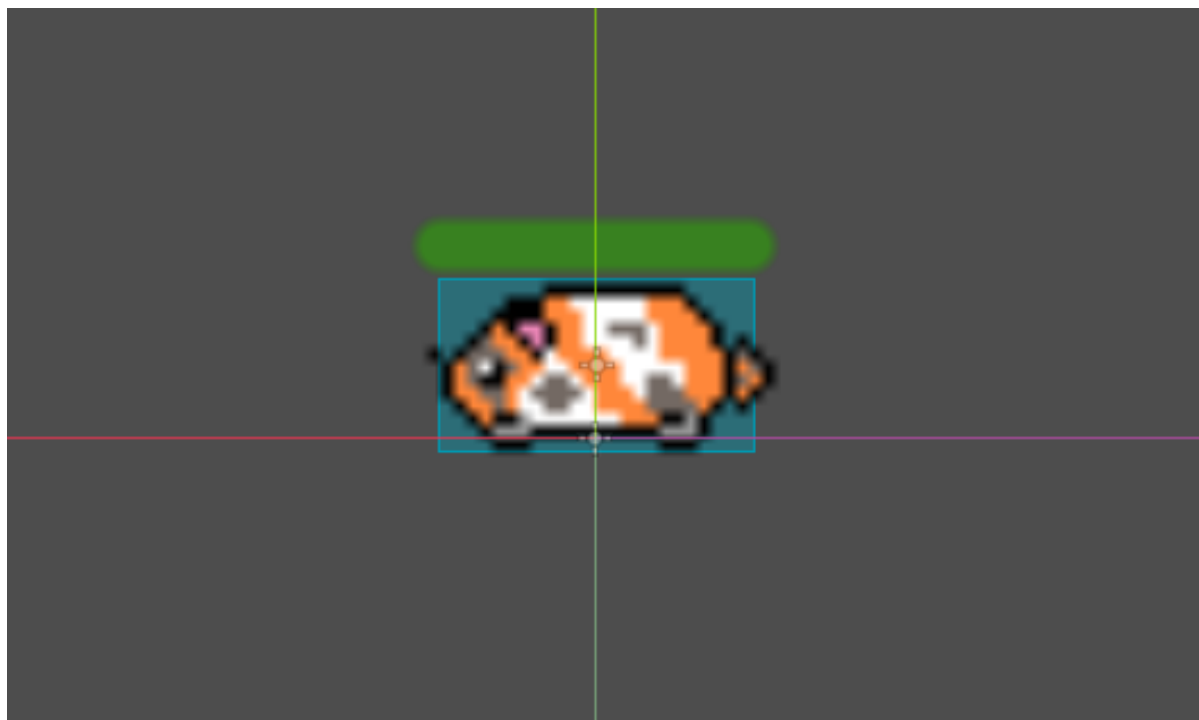
Na Rysunku 2.4 widoczny jest drugi panel menu, pozwalający na uruchomienie wskazanego poziomu gry. Po naciśnięciu przycisku **README** pojawia się panel tekstowy z informacjami o grze. Natomiast naciśnięcie **QUIT** w tym przypadku cofa użytkownika do panelu pierwszego. Struktura węzłów jest bardzo podobna do poprzedniego panelu.



Rysunek 2.4: Menu panelu drugiego

2.4. Agent

W przypadku tej pracy agentem jest chomik, który w swoim zakresie umiejętności posiada wcześniej wspomniany skok oraz ruch w prawo. Rysunek 2.5 przedstawia jego model, który został wykonany w programie GIMP 2.10.36 przez Autora pracy.



Rysunek 2.5: Model agenta

Widoczny na Rysunku 2.5 niebieski prostokąt przedstawia **kolizję** obiektu. W kontekście gier komputerowych **kolizją** jest interakcja między dwoma lub większą liczbą obiektów, w której zachodzi „kontakt fizyczny”. Zielony pasek nad agentem informuje o jego obecnym stanie życia.

Przedstawiona na Rysunku 2.6 animacja ruchu agenta składa się z trzech klatek. Szybkość animacji wynosi 5 FPS (Frames Per Second).



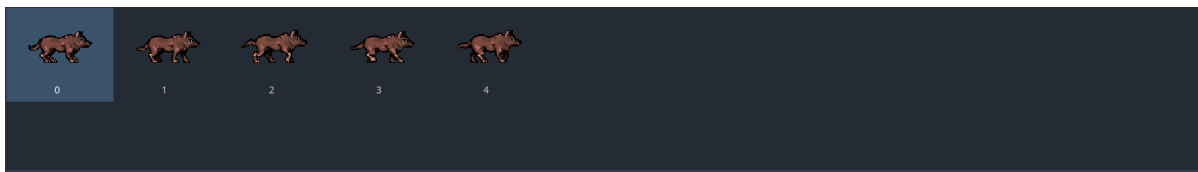
Rysunek 2.6: Animacja agenta

2.5. Przeciwnik

W grze został umieszczony tylko jeden przeciwnik – wilk. Jego model widoczny jest na Rysunku 2.7. Został wykonany przez użytkownika o pseudonimie William Thompsonj [10], wykorzystując darmową licencję. Niebieski prostokąt jest wcześniej wspomnianą **kolizją**. Natomiast mniejsze prostokąty są tak zwanymi **hitboxami**, czyli **kolizjami**, które po spotkaniu z agentem zadadzą mu obrażenia śmiertelne. Animacja ruchu wilka została przedstawiona na Rysunku 2.8.

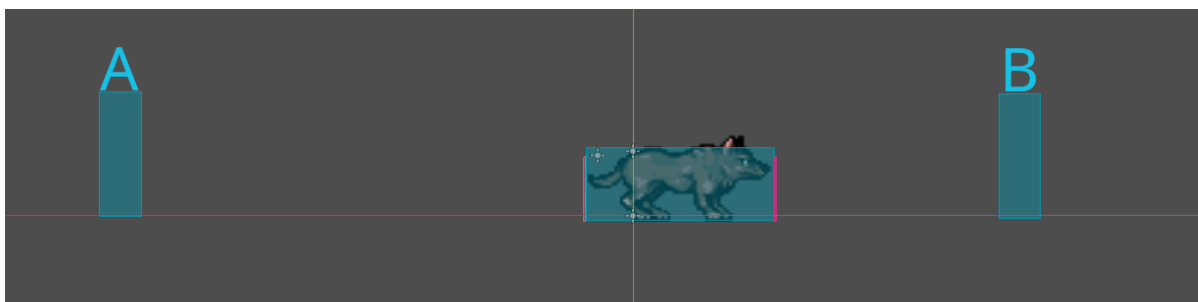


Rysunek 2.7: Model przeciwnika



Rysunek 2.8: Animacja przeciwnika

Przeciwnik został zaimplementowany w taki sposób, żeby poruszał się w środowisku od punktu A do punktu B i z powrotem w nieskończonej pętli. Zostało to pokazane na Rysunku 2.9.



Rysunek 2.9: Przeciwnik ograniczony punktami A i B

Listing 2.1 przedstawia funkcję definiującą zachowanie przeciwnika.

```
func changeDirection():
    self.directionL = !self.directionL

if is_population_dead == false:
    $AnimatedSprite2D.play("move")
    if directionL:
        $AnimatedSprite2D.flip_h = true
        velocity.x = -1 * SPEED
    else:
        $AnimatedSprite2D.flip_h = false
        velocity.x = 1 * SPEED
else:
    position = spawnPosition
    velocity.x = move_toward(velocity.x, 0, SPEED)
    directionL = true
    $AnimatedSprite2D.play("default")
```

Listing 2.1: Funkcja definiująca zachowanie przeciwnika

Flaga **directionL** odpowiada za kierunek ruchu przeciwnika i zmienia się po dojściu do odpowiedniego punktu za pomocą funkcji **changeDirection()**, która jest połączona za pomocą sygnału ze skryptem przedstawionym na Listingu 2.2.

```
func _on_body_entered(body):
    if body.has_method("changeDirection"):
        body.changeDirection()
```

Listing 2.2: Funkcja odpowiedzialna za zmianę kierunku przeciwnika

2.6. Przeszkody

W grze zostały zaimplementowane przeszkody takie jak lawa, kolce, czarna materia oraz urządzenie tworzące kolumnę lawy. Agent po spotkaniu z jakąkolwiek z nich otrzyma obrażenia śmiertelne. Rysunki oraz animacje przeszkód zostały wykonane w programie GIMP 2.10.36 przez Autora. Większość przeszkód została utworzona na bazie **tilesetów**, czyli zestawów kafelków o określonej wielkości, służących do umieszczenia na **tilemapie** przedstawionej na Rysunku 2.10. Mapa składa się z kafelków o rozmiarze 32x32 piksele.



Rysunek 2.10: Tilemap

2.6.1. Lawa

Przedstawiony na Rysunku 2.11 **tileset** składa się z 16 kafelków o rozmiarze 32x32 piksele. W każdym kafelku mapy może zostać umieszczony tylko jeden z dostępnych kafelków **tilesetu**, ale w czasie trwania gry jest on bardzo szybko zmieniany, tworząc tym sposobem animację.



Rysunek 2.11: Tileset lawy

2.6.2. Czarna materia

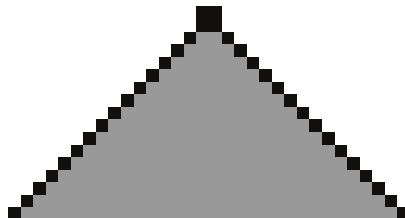
Tileset czarnej materii został stworzony na bazie **tilesetu** lawy. Różni się przede wszystkim kolorystycznie. Przedstawiony jest na Rysunku 2.12.



Rysunek 2.12: Tileset czarnej materii

2.6.3. Kolce

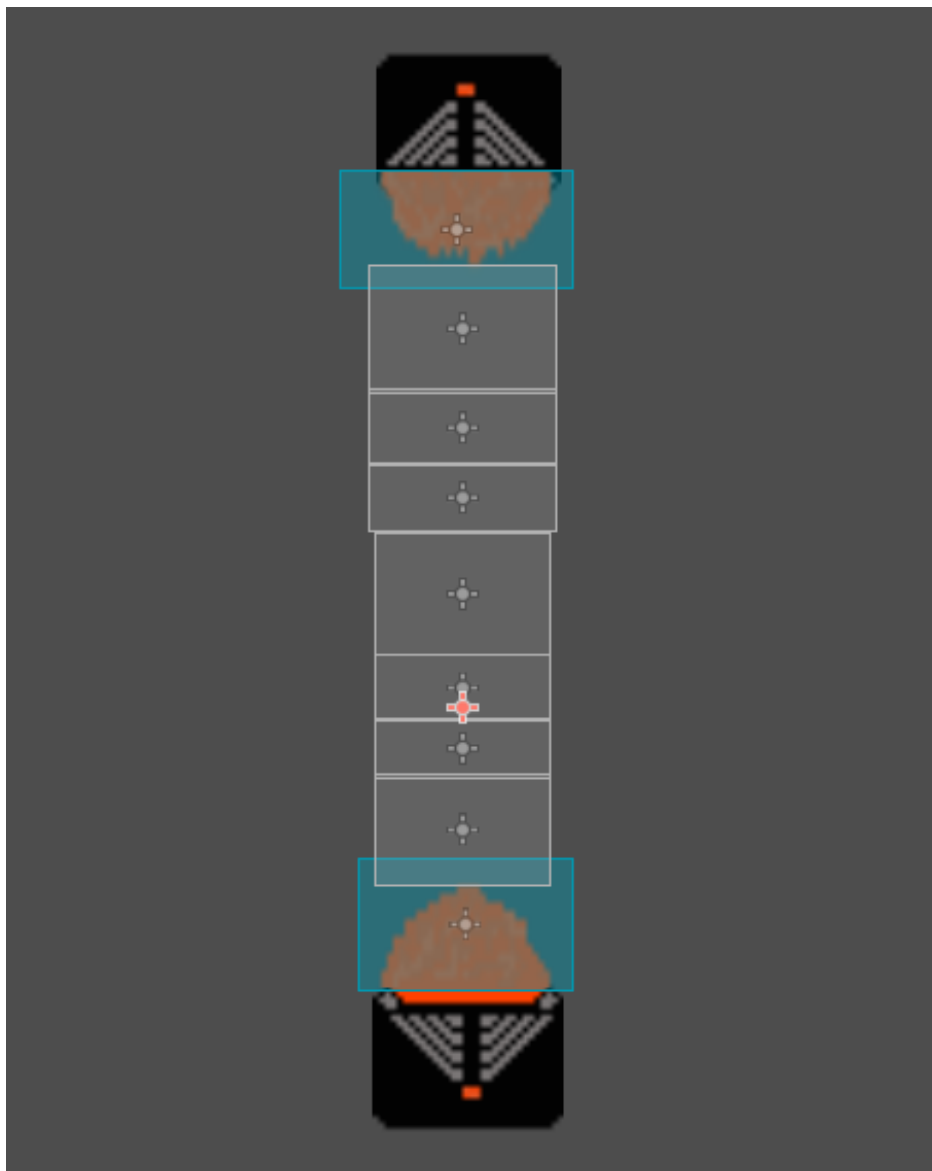
Przedstawiony na Rysunku 2.13 **tileset** kolców składa się tylko z jednego kafelka i nie posiada animacji.



Rysunek 2.13: Tileset kolców

2.6.4. Emiter lawy

W odróżnieniu od pozostałych przeszkód, emiter nie jest **tilesetem**. Składa się on z dwóch animowanych grafik (kolumna lawy i emiter) oraz **kolizji**. Szare prostokąty są **hitboxami**, które uaktywniają się w zależności od stanu animacji i w ich miejsce pojawiają się odpowiednie klatki kolumny lawy. Natomiast grafika znajdująca się przy niebieskim prostokącie służy do przedstawienia jak kolumna lawy jest tworzona. Zostało to zaprezentowane na Rysunku 2.14.



Rysunek 2.14: Scena emitera lawy

Animacja kolumny lawy została przedstawiona na Rysunku 2.15. W zależności od wyświetlanych w danym czasie klatek uaktywniają się wcześniej wspomniane **hitboxy**. Agent jest w stanie ominąć tę przeszkodę, jeżeli wykona skok w odpowiednim czasie.



Rysunek 2.15: Animacja kolumny lawy

Ostatnia animacja emitera lawy została pokazana na Rysunku 2.16.



Rysunek 2.16: Animacja emitera lawy

Poszczególne animacje emitera lawy są zarządzane przez węzeł **Animation-Player**, umożliwiający zmianę atrybutów i sterowanie animacją. Zostało to pokazane na Rysunku 2.17. Animacja trwa 6 sekund.



Rysunek 2.17: AnimationPlayer

2.7. Kamera

Godot w swojej hierarchii posiada węzeł **Camera2D**, który po odpowiedniej modyfikacji umożliwia poruszanie się po środowisku za pomocą kursora. Ponadto zmodyfikowana kamera posiada opcje przybliżania oraz oddalania. Implementacja została pokazana na Listingu 2.3.

```
var dragging = false
var is_onto_console = false

func _process(_delta):
    if is_onto_console == false:
        if Input.is_action_pressed("B_LEFT"):
            dragging = true
        else:
            dragging = false
        if Input.is_action_pressed("B_LEFT") and dragging:
            position = get_global_mouse_position()

    if Input.is_action_just_pressed("S_UP"):
        if zoom.x + 1 > 6 and zoom.y + 1 > 6:
            self.zoom.x = 6
            self.zoom.y = 6
        else:
            self.zoom.x += 1
            self.zoom.y += 1
    if Input.is_action_just_released("S_UP"):
        position = get_global_mouse_position()
```

```

elif Input.is_action_just_pressed("S_DOWN"):
    if zoom.x - 1 <= 0 and zoom.y - 1 <= 0:
        zoom.x = 1
        zoom.y = 1
    else:
        self.zoom.x -= 1
        self.zoom.y -= 1

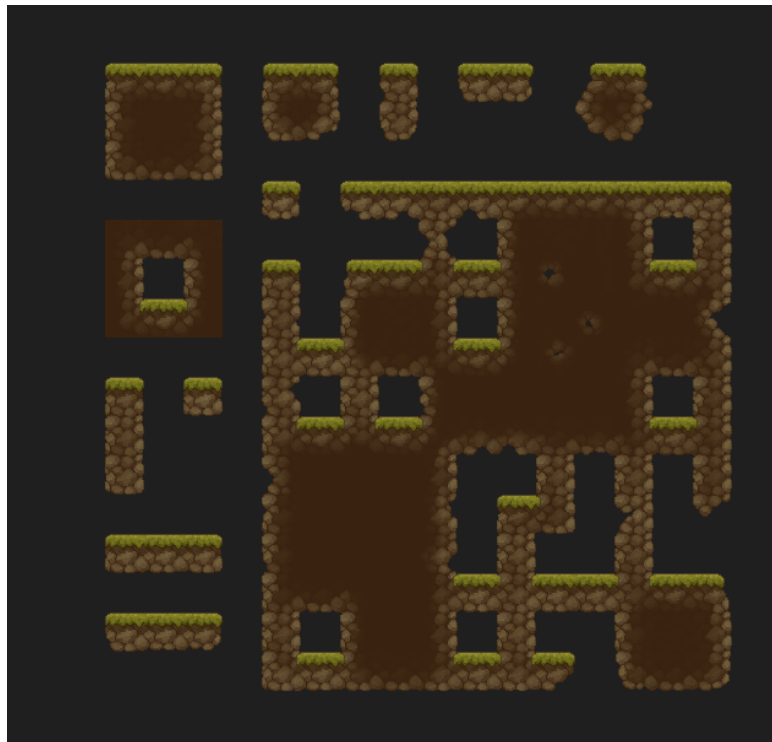
```

Listing 2.3: Funkcja definiująca działanie kamery

Po naciśnięciu lewego przycisku myszy (**B_LEFT**) nastąpi przeniesienie kamery do punktu wskazanego przez kursor pod warunkiem, że nie będzie on znajdował się na konsoli (flaga **is_onto_console**). Natomiast poruszanie kółkiem myszy będzie odpowiednio przybliżać oraz oddalać kamerę.

2.8. Poziomy

W grze zostały umieszczone cztery poziomy stworzone na bazie dwóch **tilesetów** przedstawionych na Rysunku 2.18 i 2.19 oraz wcześniej omówionych elementów takich jak przeszkody czy przeciwnik. Przedstawione **tilesety** zostały utworzone przez użytkownika o pseudonimie Cainos [11].



Rysunek 2.18: Tileset platform



Rysunek 2.19: Tileset dodatków estetycznych

2.8.1. Poziom 0

Początkowy poziom (numer 0) jest najłatwiejszy oraz najkrótszy. Został umieszczony w nim jeden przeciwnik oraz kilka platform. W przypadku nieudanego skoku między platformami agent wpada do czarnej materii. Cały poziom został przedstawiony na Rysunku 2.20.

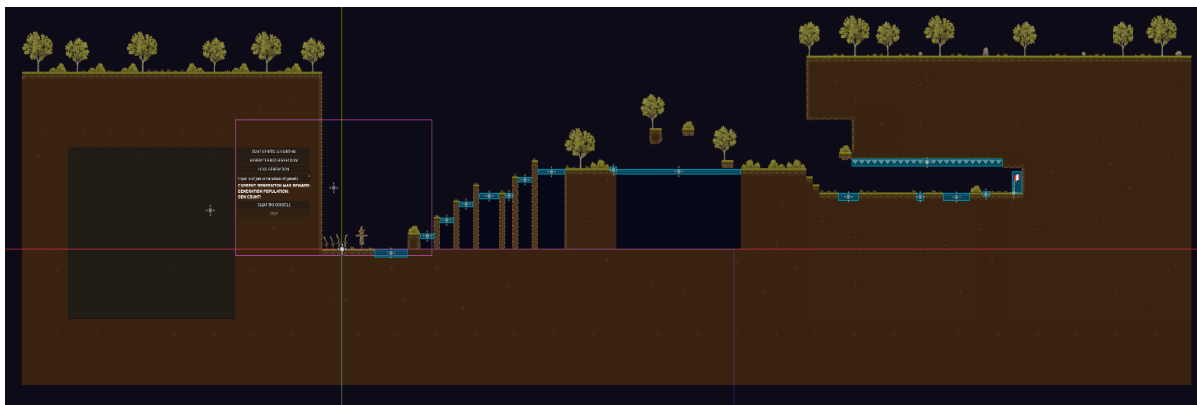


Rysunek 2.20: Poziom 0

2.8.2. Poziom 1

W poziomie pierwszym (numer 1) pojawiają się przeszkody takie jak czarna materia oraz kolce. Ponadto poziom ten wprowadza nowy element taki jak

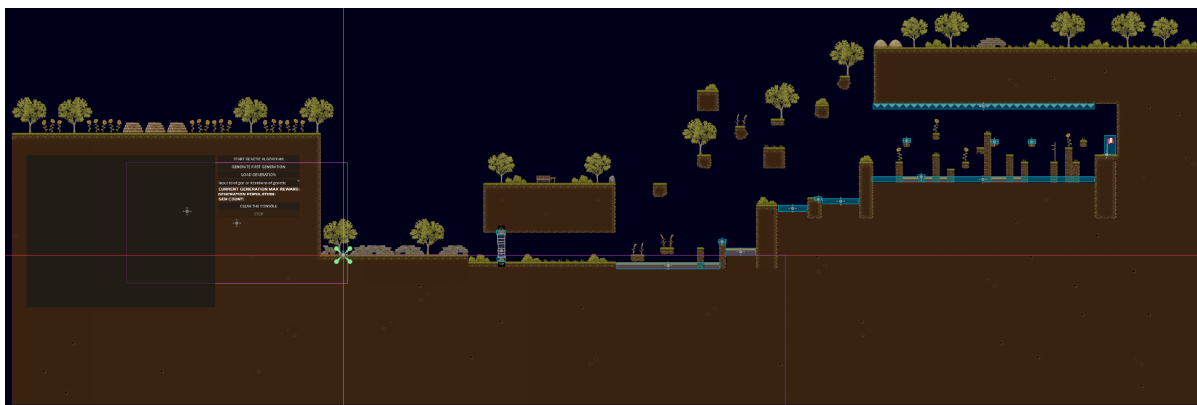
trampolina, która wyrzuca agenta na znaczną wysokość. Całość przedstawia Rysunek 2.21.



Rysunek 2.21: Poziom 1

2.8.3. Poziom 2

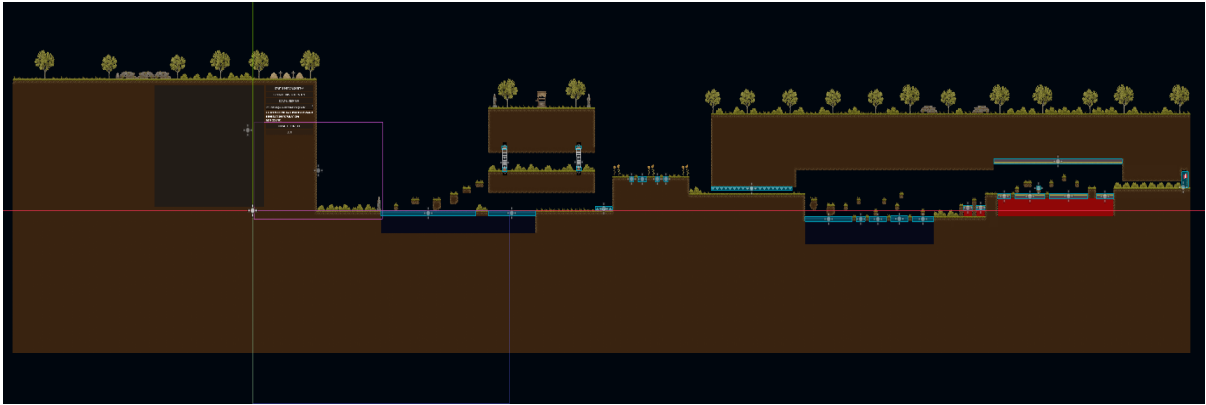
W poziomie drugim (numer 2) po raz pierwszy pojawia się emiter lawy. Ponadto można tam napotkać przeszkody takie jak kolce, lawa oraz czarna materia. Przedstawia to Rysunek 2.22.



Rysunek 2.22: Poziom 2

2.8.4. Poziom 3

Poziom trzeci (numer 3) jest najdłuższy i można w nim napotkać przeszkody takie jak emiter lawy, lawa, kolce oraz czarna materia. Całość została pokazana na Rysunku 2.23.



Rysunek 2.23: Poziom 3

2.9. Implementacja algorytmu

W celu nauczenia agenta przechodzenia poziomów gry platformowej wykorzystano zmodyfikowany algorytm genetyczny połączony z uczeniem maszynowym ze wzmocnieniem. Dodatkowo został utworzony menadżer zarządzający procesem nauki agentów. Podczas uruchomienia jakiegokolwiek poziomu wykona się kod przedstawiony na Listingu 2.4.

```
for i in range(0, gen_population):
    var p = preload("res://Scenes/player.tscn").instantiate()
    get_parent().call_deferred("add_child",p)
    set_player_configuration(p,i)
    p.t = Thread.new()

manager = Thread.new()
manager.start(call,Thread.PRIORITY_HIGH)
```

Listing 2.4: Konfiguracja początkowa algorytmu

Przedstawiony kod tworzy w środowisku liczbę agentów zależną od zmiennej **gen_population**, a następnie ustawia ich konfigurację (indeks osobnika, losowy kolor czy miejsce odrodzenia). Potem uruchomiany jest wątek zarządzający procesem nauki, którego kod został przedstawiony na Listingu 2.5.

```
func learn():
    while true:
        generate_first_gen_procedure()
        load_generation_procedure()
        start_genetic_procedure()

        if is_population_dead() and genetic_iter < iter_saved:
            find_the_best_player_and_generate_population()
            start_living_process()

        await get_tree().create_timer(0.01).timeout
```

Listing 2.5: Funkcja zarządzająca procesem nauki agentów

Trzy funkcje pod pętlą **while** odpowiadają za wykonanie konkretnych instrukcji, których działanie wymaga naciśnięcia odpowiedniego przycisku. Najważniejsza część kodu znajduje się pod wspomnianymi funkcjami. Jeżeli populacja wyginie oraz obecna iteracja będzie mniejsza od ilości iteracji wprowadzonych przez użytkownika,

nastąpi proces selekcji osobników, który został pokazany na Listingu 2.6. Konieczne jest wygenerowanie pierwszej generacji, żeby selekcja ta była możliwa.

```
func find_the_best_player_and_generate_population():
    load_players_data(path_to_genetic + "gen" + str(gen_last) + ".txt")
    players.sort_custom(compare_p1_p2_reward)

    var steps = gen_population / 2
    var index = 0
    var best_players_move_array = players[0].moves

    for i in range(0, steps):
        players[index].moves = players[i].moves.duplicate()
        for pop_counter in range(0,randi_range(1,3)):
            players[index].moves.pop_back()
            index += 1

    for i in range(0, steps):
        players[index].moves = best_players_move_array.duplicate()
        for pop_counter in range(0,randi_range(1,3)):
            players[index].moves.pop_back()
            index += 1
```

Listing 2.6: Funkcja definiująca selekcję

Przedstawiony kod selekcji polega na tym, że sortowana jest tablica agentów w zależności od liczby zebranych punktów. Następnie generowana jest nowa populacja: pierwsza połowa nowej populacji jest tworzona na podstawie ruchów najlepszych graczy z poprzedniej populacji, a druga połowa – na podstawie ruchów najlepszego gracza. W obu przypadkach, część ruchów jest usuwana w sposób losowy w zakresie 1-3, aby wprowadzić pewną zmienność w nowej populacji. Zrezygnowano z elementu krzyżowania oraz mutacji, ponieważ wykonane ruchy są bardzo ważne. Mutacja może doprowadzić do śmierci agenta w momencie, w którym powinien jeszcze być zdolnym do wykonywania ruchów. Natomiast krzyżowanie wymagałoby wyrównywania tablic ruchów agentów, co także może doprowadzić do błędnego działania. Po utworzeniu nowej populacji następuje proces nauki agentów (**start living process**), który dla każdego agenta uruchamia wątek (1 agent = 1 wątek) wykonujący funkcję przedstawioną na Listingu 2.7.

```
func live():
    if is_dead == false:
        if moves.is_empty() == false:
            for move in moves:
                previous_pos_x = pos_x
                move(move)
                await get_tree().create_timer(2.25).timeout
                move_end_pos_x = pos_x

                if move_end_pos_x > previous_pos_x:
                    move_reward(move)

            while true:
                var move = randi_range(1,6)
                previous_pos_x = pos_x
                moves.append(move)

                move(move)
                await get_tree().create_timer(2.25).timeout

                move_end_pos_x = pos_x

                if move_end_pos_x > previous_pos_x:
                    distance_after_death += abs(respawnPosition.distance_to(Vector2(
                        move_end_pos_x, 0)))
```

```

        move_reward(move)
        iter_stacked_counter = 0
    else:
        iter_stacked_counter += 1
        reward -= 5
        if iter_stacked_counter == 2:
            for i in range(0,2):
                moves.pop_back()
                reward -= 375
                respawn()

    if is_dead == True:
        return
    return

```

Listing 2.7: Funkcja definiująca zachowanie agenta

Początkowo agent wykonuje ruchy, które zostały mu przekazane przez jego poprzedników (jeżeli nie jest to pierwsza generacja). Następnie wykonuje losowe ruchy, aż do momentu swojej śmierci bądź dostania się do punktu końcowego gry. Gdy to nastąpi flaga **is_dead** zmienia swoją wartość na **true**, a agent wraca na miejsce odrodzenia i czeka na rozpoczęcie kolejnej iteracji. Za każdy ruch agent może otrzymać nagrodę lub karę (funkcja **move_reward()**). W sytuacji, gdy agent nie będzie mógł poruszać się dalej lub zginie, sprawdzana jest jego poprzednia i nowa pozycja na osi x układu współrzędnych. Gdy licznik **iter_stacked_counter** będzie równy 2, nastąpi śmierć agenta. Agent traci 300 punktów za każdy zgon. Jeżeli agent nie może przejść dalej, odejmowane jest 375 punktów.

Implementacja funkcji ruchu (**move()**) została przedstawiona na Listingu 2.8.

```

func move(move : int):
    if move == 0:
        is_moving_right = true
        await get_tree().create_timer(0.1).timeout
        is_moving_right = false
    elif move == 1:
        is_moving_right = true
        await get_tree().create_timer(0.25).timeout
        is_moving_right = false
    elif move == 2:
        is_moving_right = true
        await get_tree().create_timer(0.5).timeout
        is_moving_right = false
    elif move == 3:
        is_jumping = true
        is_moving_right = true
        await get_tree().create_timer(0.1).timeout
        is_moving_right = false
    elif move == 4:
        is_jumping = true
        is_moving_right = true
        await get_tree().create_timer(0.25).timeout
        is_moving_right = false
    elif move == 5:
        is_jumping = true
        is_moving_right = true
        await get_tree().create_timer(0.5).timeout
        is_moving_right = false
    elif move == 6:
        is_jumping = true
        is_moving_right = true
        await get_tree().create_timer(0.75).timeout
        is_moving_right = false

```

Listing 2.8: Funkcja definiująca ruch agenta

Funkcja **move()** w zależności od podanej w argumencie liczby wykonuje określony ruch poprzez odpowiednie ustawienie flagi **is_jumping** lub **is_moving_right**. Każdy ruch posiada swój czas trwania (**await get_tree().create_timer()**). Liczby od 0 do 2 oznaczają wyłącznie ruch w prawo. Natomiast liczby od 3 do 6 oznaczają ruch w prawo oraz skok.

Funkcja nagrody (**move_reward**) została przedstawiona na Listingu 2.9.

```
func move_reward(move : int):
    if move == 0:
        reward += 5
    elif move == 1:
        reward += 25
    elif move == 2:
        reward += 50
    elif move == 3:
        reward += 5
    elif move == 4:
        reward += 25
    elif move == 5:
        reward += 50
    elif move == 6:
        reward += 65
```

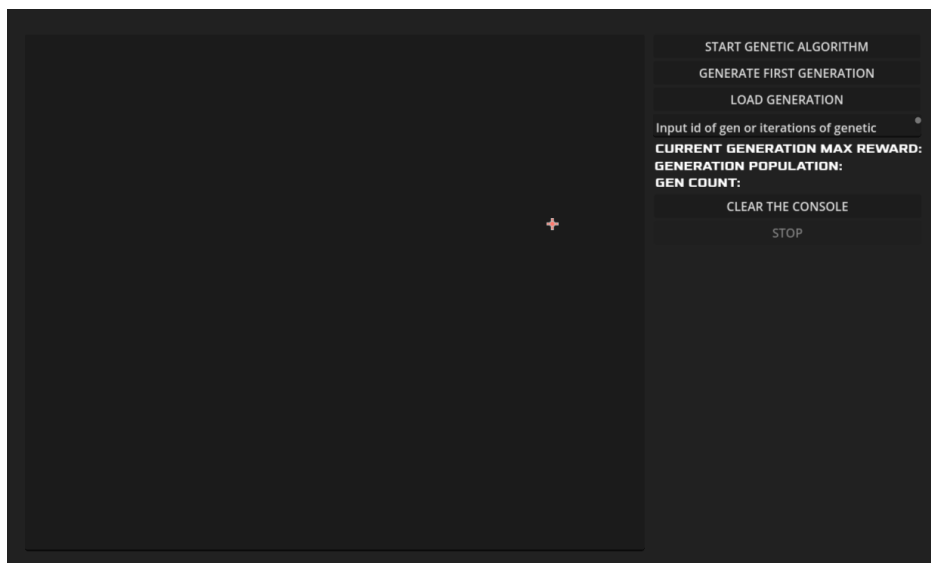
Listing 2.9: Funkcja nagrody

Podsumowując, algorytm genetyczny służy do stworzenia populacji agentów. Uczenie maszynowe ze wzmocnieniem występuje w momencie, gdy agent wykonuje losowy ruch za pomocą funkcji **move()**, otrzymując za to nagrodę lub karę (**move_reward**) w zależności od użytej strategii. Proces ten jest iteracyjny i opiera się na metodzie prób i błędów, gdzie agent stopniowo uczy się przechodzić konkretny poziom gry.

W celu efektywnego działania algorytmu liczba agentów jest uzależniona od dostępnej liczby procesorów logicznych w komputerze. Takie podejście zostało zastosowane do zarządzania agentami przy użyciu programowania współbieżnego, gdzie każdy agent jest obsługiwany przez dedykowany wątek, umożliwiając im jednoczesne wykonywanie ruchów.

2.10. Konsola

W celu odpowiedniego zarządzania algorytmem do gry została dodana konsola pokazana na Rysunku 2.24.



Rysunek 2.24: Konsola

Przycisk **START GENETIC ALGORITHM** rozpoczyna proces nauczania agentów i trwa określoną liczbę iteracji podanych przez użytkownika w polu tekstowym znajdującym się pod przyciskiem **GENERATE FIRST GENERATION**.

Przycisk **GENERATE FIRST GENERATION** służy do stworzenia pierwszej generacji, na podstawie której będą się uczyć przyszli agenci.

LOAD GENERATION służy do wczytania i wykonania określonych ruchów zawartych w podanej przez użytkownika generacji w polu wprowadzania danych, której indeksowanie zaczyna się od 1.

Przycisk **CLEAR THE CONSOLE** służy do wyczyszczenia pola znajdującego się po lewej stronie, w którym są umieszczane informacje takie jak tablica ruchów agenta, nagroda oraz komunikaty o błędach.

Przycisk **STOP** anuluje działanie procesu nauczania agentów pod warunkiem, że obecna populacja wyginie bądź dostanie się do mety.

Rozdział 3

Testowanie aplikacji

W ostatnim rozdziale zostały przedstawione wyniki przeprowadzonych testów dla poszczególnych poziomów.

3.1. Scenariusz testów

Testy polegały na sprawdzeniu, jaki wpływ posiada liczebność populacji na liczbę generacji potrzebnych do ukończenia poziomu. Ponadto analizie została poddana liczba ruchów najlepszego agenta (osobnik, który jako pierwszy dostał się do punktu końcowego). Dodatkowo każdy poziom był obserwowany pod względem miejsc, które były najbardziej problematyczne dla agentów. Każdy test dla poszczególnej populacji był powtarzany 5 razy. Następnie na podstawie zebranych danych została policzona średnia liczba generacji, średnia liczba ruchów, minimum oraz maksimum generacji, dla każdej z badanych liczebności populacji.

3.2. Testy

Test poziomu 0 przedstawia dane zawarte w Tabeli 3.1. Wyniki pokazują, że średnia liczba ruchów najlepszego osobnika nie zależy od liczebności populacji. Wystarczy 2 do 4 generacji dla populacji liczącej 256 osobników, aby ukończyć poziom gry.

Populacja	Średnia liczba generacji	Średnia liczba ruchów	Min. generacji	Max. generacji
16	8	17	4	10
32	5	18	3	7
64	6	19	2	9
128	4	17	2	6
256	3	18	2	4

Tabela 3.1: Test poziom 0

Test poziomu 1 przedstawia Tabela 3.2. Wraz ze wzrostem populacji (na przykład z 16 do 256 agentów), średnia liczba generacji zazwyczaj maleje. Na przykład, dla 16

agentów średnia liczba generacji wynosi 13, podczas gdy dla 256 agentów spada do 5. Może to sugerować, że większa liczba agentów przyspiesza rozwiązanie problemu.

Populacja	Średnia liczba generacji	Średnia liczba ruchów	Min. generacji	Max. generacji
16	13	26	9	21
32	10	27	9	12
64	11	24	7	14
128	7	30	6	8
256	5	31	5	6

Tabela 3.2: Test poziom 1

Test poziom 2 przedstawia Tabela 3.3. Wyniki pokazują, że w tym przypadku liczebność populacji nie miała znaczącego wpływu na przejście poziomu gry. Świadczy o tym fakt, że różnice między średnią liczbą generacji dla populacji: 32, 64, 128, 256 były niewielkie.

Populacja	Średnia liczba generacji	Średnia liczba ruchów	Min. generacji	Max. generacji
16	13	29	9	18
32	8	28	6	10
64	9	30	6	13
128	8	30	6	12
256	6	28	5	8

Tabela 3.3: Test poziom 2

Test poziom 3 przedstawia Tabela 3.4. Wyniki pokazują, że mniejsze populacje (na przykład 16, 32, 64) potrzebowały stosunkowo dużej liczby generacji, aby przejść dany poziom w porównaniu do większych populacji.

Populacja	Średnia liczba generacji	Średnia liczba ruchów	Min. generacji	Max. generacji
16	30	49	22	42
32	21	47	17	28
64	17	50	15	20
128	9	49	7	12
256	9	49	8	11

Tabela 3.4: Test poziom 3

3.3. Wnioski

W wyniku przeprowadzonych testów oraz obserwacji algorytmu zostały wyciągnięte następujące wnioski:

- Większe populacje są zdecydowanie bardziej efektywne (efektywne w znaczeniu, że w celu przejścia poziomu większe populacje potrzebują mniejszej liczby ruchów i generacji), a świadczy o tym fakt, że liczba generacji maleje wraz ze wzrostem wielkości populacji. Jednakże znaczące różnice zaczynają się od liczebności populacji równej 128 i więcej.
- Liczebność populacji nie wpływa na liczbę ruchów potrzebnych do ukończenia poziomu.
- Znacząca liczba przeszkód na danym terenie znacznie wydłuża czas przejścia tego obszaru.
- Najbardziej problematyczną przeszkodą okazały się rozległe przepaście między platformami, które można było przejść, stojąc odpowiednio na ich krawędzi.
- Algorytm omówiony w pracy nie będzie w stanie przejść każdego poziomu gier platformowych 2D. Najlepiej jest dostosować rozwiązania do konkretnego rodzaju poziomów bądź pod jeden konkretny poziom.
- Parametry pozwalające na eksperymentowanie to: liczebność populacji, czasy trwania ruchów oraz funkcja nagrody.
- Najtrudniejszym oraz najdłuższym poziomem okazał się poziom 3. Natomiast poziom 0 jest najłatwiejszy. Z kolei poziomy 2 i 3 posiadają podobny poziom trudności.

Podsumowując, zastosowane podejście umożliwiło przechodzenie poziomów gry platformowej 2D. Algorytm można byłoby ulepszyć na przykład poprzez dodanie sieci neuronowej, która w tym przypadku analizowałaby obraz gry i przewidywałaby możliwe ruchy dla danego stanu.

Podsumowanie

Celem tej pracy było utworzenie gry platformowej 2D, w której agent za pomocą algorytmu uczenia maszynowego mógł przechodzić poziomy gier komputerowych.

Wykorzystany algorytm pozwolił między innymi wykryć błędy w źle ustawionych **kolizjach**, czy też pokazać rozwiązania niekoniecznie oczywiste.

Testy poziomów wykazały, że zaproponowany algorytm nie jest uniwersalnym rozwiązaniem do przejścia każdego poziomu gier platformowych 2D. Zalecane jest dostosowanie strategii do konkretnego rodzaju poziomów lub nawet do jednego specyficznego przypadku.

Wykonana gra może stanowić bazę do dalszego testowania. Nawet drobne zmiany takie jak dodanie nowego przeciwnika o specyficznym zachowaniu mogą doprowadzić do interesujących rezultatów.

Bibliografia

- [1] *Co to są agenci sztucznej inteligencji?*, <https://learn.microsoft.com/pl-pl/azure/cloud-adoption-framework/innovate/best-practices/conversational-ai> (dostęp: 14.02.2024).
- [2] *Understanding PEAS in Artificial Intelligence*, <https://www.geeksforgeeks.org/understanding-peas-in-artificial-intelligence> (dostęp: 14.02.2024).
- [3] Stuart Russell, Peter Norvig, *Artificial Intelligence: A Modern Approach*, Wydawnictwo Pearson, 1995.
- [4] Aleksandra Król-Nowak, Katarzyna Kotarba, *Podstawy uczenia maszynowego*, Wydawnictwo AGH, 2022.
- [5] *Co łączy psychologię i sztuczną inteligencję?*, <https://web.swps.pl/strefa-psyche/blog/relacje/22201-co-laczy-psychologie-i-sztuczna-inteligencje?dt=1707951215000> (dostęp: 14.02.2024).
- [6] *Co to jest uczenie się przez wzmacnianie?*, <https://www.unite.ai/what-is-reinforcement-learning> (dostęp: 14.02.2024).
- [7] *Q-Learning*, <https://datascience.eu/pl/uczenie-maszynowe/q-learning> (dostęp: 14.02.2024).
- [8] A.E. Eiben, J.E. Smith, *Introduction to Evolutionary Computing*, Wydawnictwo Springer, 2003.
- [9] *Tło menu*, <https://wallpapercave.com/w/wp11630931> (dostęp: 14.02.2024).
- [10] *Model wilka*, <https://opengameart.org/content/lpc-wolf-animation> (dostęp: 14.02.2024).
- [11] *Tileset terenu oraz dodatków dekoracyjnych*, <https://cainos.itch.io/pixel-art-platformer-village-props> (dostęp: 14.02.2024).

Spis listingów

2.1.	Funkcja definiująca zachowanie przeciwnika	14
2.2.	Funkcja odpowiedzialna za zmianę kierunku przeciwnika	14
2.3.	Funkcja definiująca działanie kamery	18
2.4.	Konfiguracja początkowa algorytmu	22
2.5.	Funkcja zarządzająca procesem nauki agentów	22
2.6.	Funkcja definiująca selekcję	23
2.7.	Funkcja definiująca zachowanie agenta	23
2.8.	Funkcja definiująca ruch agenta	24
2.9.	Funkcja nagrody	25

Spis tabel

3.1. Test poziom 0	27
3.2. Test poziom 1	28
3.3. Test poziom 2	28
3.4. Test poziom 3	28

Spis rysunków

1.1.	Mario (źródło: https://blogs.loc.gov/now-see-hear/2023/08/from-the-national-recording-registry-super-mario-1985) . .	4
1.2.	Początkowy poziom gry Mario (źródło: https://www.reddit.com) . .	4
1.3.	Elementy uczenia maszynowego przez wzmacnianie (źródło: https://mirosławmamczur.pl/czym-jest-uczenie-maszynowe-i-jakie-sa-rodzaje)	6
2.1.	Główne okno Godot	10
2.2.	Menu panelu początkowego	10
2.3.	Zrzut ekranu przedstawiający węzły menu panelu początkowego . . .	11
2.4.	Menu panelu drugiego	12
2.5.	Model agenta	12
2.6.	Animacja agenta	13
2.7.	Model przeciwnika	13
2.8.	Animacja przeciwnika	14
2.9.	Przeciwnik ograniczony punktami A i B	14
2.10.	Tilemap	15
2.11.	Tileset lawy	15
2.12.	Tileset czarnej materii	16
2.13.	Tileset kolców	16
2.14.	Scena emitera lawy	17
2.15.	Animacja kolumny lawy	17
2.16.	Animacja emitera lawy	18
2.17.	AnimationPlayer	18
2.18.	Tileset platform	19
2.19.	Tileset dodatków estetycznych	20
2.20.	Poziom 0	20
2.21.	Poziom 1	21
2.22.	Poziom 2	21
2.23.	Poziom 3	22
2.24.	Konsola	26